2

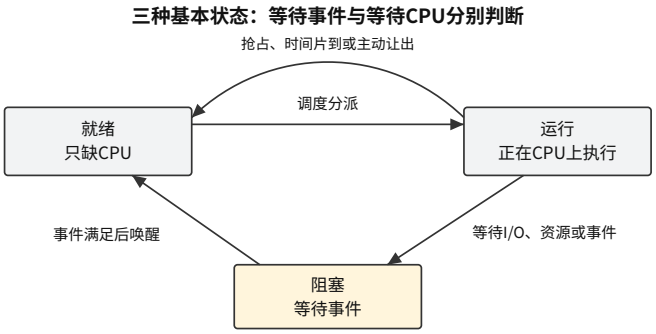


Figure 2.1: 运行、就绪、阻塞的转换条件

2.1 进程实体、PCB 与线程

进程是程序的一次执行过程及其资源环境，是资源分配和管理的基本单位。进程实体由程序段、数据段和 PCB 构成；PCB 是进程存在和管理的标志，记录标识、状态、调度信息、资源与处理器现场。进程具有动态性、并发性、独立性、异步性和结构性，动态性是最基本特征。

对象	共享与独有内容
不同进程	各有 PCB 与地址空间；通信共享须按机制建立
同进程线程	共享代码、数据、堆、打开文件等进程资源
每条线程	独有 TID、PC、寄存器现场、栈、运行状态、优先级等 TCB 信息

支持内核级线程时，内核按可调度线程分配处理器；纯用户级线程由线程库调度，内核仍调度其承载实体。同进程线程的栈用于各自调用链，其他线程取得有效地址后仍可能访问它。父子进程各有 PCB、可并发执行；写时复制可暂时共享物理页，逻辑地址空间仍分别管理。

2.2 状态转换与进程控制

转换	条件
就绪 → 运行	被调度并获得 CPU
运行 → 就绪	时间片到期、被抢占、主动让出 CPU 且仍具执行条件
运行 → 阻塞	等待 I/O、资源或同步事件
阻塞 → 就绪	对应等待条件满足，执行唤醒

就绪仅缺 CPU；阻塞还缺事件。单 CPU 同一时刻至多一个相应实体运行。阻塞者须先就绪再运行；就绪者未执行，不能主动发起新的阻塞操作。

七状态模型在创建、就绪、运行、阻塞、终止上增加就绪挂起和阻塞挂起。经典模型的挂起表示映像暂不驻留内存，PCB 继续保留；阻塞挂起

者事件满足后转为就绪挂起；激活后分别进入就绪或阻塞。运行者也可被挂起，创建者可按策略进入就绪或就绪挂起。

例 2-1

下列由当前线程引起的事件或执行的操作中，可能导致该线程由执行态变为就绪态的是（ ）。

A. 键盘输入　　B. 缺页异常　　C. 主动出让 CPU　　D. 执行信号量的 wait() 操作

控制原语	必须一致更新的内容
创建	申请标识、PCB 与资源，初始化现场，加入就绪等队列
撤销	停止执行、处理通知及等待关系、回收资源与 PCB；子进程政策依系统
阻塞	保存现场、改状态、进入对应事件等待队列并调度
唤醒	移出等待队列、改为就绪、加入就绪队列
切换	保存旧现场、更新状态队列、恢复新现场

PCB 可按状态链接成队列，或由索引表定位；阻塞队列可按事件细分。状态字段与队列关系必须原子维护；进程阻塞后的长时间等待允许 CPU 调度其他进程。

2.3 用户级、内核级线程与映射

模型	管理与调度	阻塞及并行边界
多对一	用户线程库管理多个 ULT，内核仅见一个承载实体	承载实体阻塞会挡住全部；这些线程不能多核并行
一对一	每个用户线程对应一个 KLT，由内核分别调度	一个阻塞时其他可运行，可多核并行；管理成本较大
多对多	用户线程复用若干 KLT，库与内核分层调度	并行度受可用 KLT 与 CPU 核数约束

纯用户级切换可不进入内核，开销较小。内核级描述管理方式，应用代码仍可在用户态执行。创建、切换开销随实现而变。

2.4 进程通信、管道与信号

机制	数据或事件的组织	使用条件
共享内存	多个虚拟映射指向共同物理区，建立后普通访存交换数据	共享布局与访问同步由通信方约定
消息传递	直接指定进程，或经信箱／端口间接通信	发送、接收是否阻塞由接口规定
管道	有限容量、顺序消费的数据流；经典无名管道为半双工、常用于亲缘进程	双向可用两条管道；命名管道可按名称联系
信号	通知事件，可采用默认动作或获准的用户处理程序	普通信号可能合并；部分信号不可捕获、忽略或屏蔽

共享内存可按规定数据结构组织，也可提供存储区由进程约定布局。管道空时阻塞读、满时阻塞写；有数据即可读，有空间即可写，无须等待整管装满或读空。读出消耗数据，多读者取得不同数据；字节流通常不保留应用记录边界，原子写长度和请求大小按接口。

普通信号位向量模型： `pending` 记录待处理，`blocked` 记录暂缓递送，当前可递送集合

$$D = \text{pending} \ \& \ \sim \text{blocked}.$$

屏蔽信号不使整个进程进入阻塞态。递送可发生在返回用户态等适当时机，用户处理程序一般在用户态执行。信号通知、信号量许可、处理器异常分属不同机制；合法缺页可由内核修复而不递送用户信号。

RR 首次响应上界

在时间片边界有 n 个就绪进程（含待考察者）、忽略切换开销，且新到达者不插入其前方时，时间片为 q 的轮转使该进程至多等待 $(n-1)q$ 后首次运行。若观察时刻处于片内，还须计当前片的剩余部分及其前方队列。

空时阻塞读、满时阻塞写

以 Linux 阻塞式管道为例：空管道在仍有写端打开时等待；全部写端关闭且已有数据读尽后，read 返回 0，表示 EOF。存在读端且一次写入的字节数 0<n≤PIPE_BUF 时，要求整体原子写入，剩余空间不足 n 字节仍会等待，因此“有空间即可写”还须核对本次请求大小。fork 后若保留不用的写端，读者可能一直等不到 EOF。

2.5 调度层次、时机与指标

层次	选择或管理的对象
高级／作业调度	后备作业进入内存并取得运行资格
中级／内存调度	换出、换入，调节驻留进程数量
低级／进程调度	从就绪执行实体中分配 CPU

结束、主动阻塞或让出 CPU 时须选择可运行者；新到达、唤醒、时钟到期可触发抢占判断。非抢占策略让当前执行者运行到结束或主动放弃；抢占策略可在规定事件点改选。

调度器选择执行者，分派器恢复现场并交付 CPU。一次中断、一次调度条件检查、一次实际上下文切换须分别计数。内核管理队列的原子临界操作期间限制调度；持有普通资源互斥权的进程仍可能被切换。无普通就绪者时运行最低优先级的 idle 路径，保留中断响应能力。

设到达 A_i、首次运行 S_i、完成 C_i、CPU 服务 B_i，则

$$T_i = C_i - A_i, \quad R_i = S_i - A_i, \quad T_i^{\mathrm{w}} = T_i / B_i.$$

仅一个 CPU 段、无 I/O、忽略切换成本时，等待 $W_i = T_i - B_i$；有 I/O 时还须扣去阻塞时间。各指标逐项计算后取算术平均，平均带权周转通常无法由两项平均数相除得到。吞吐量为完成任务数／观察时长，利用率为设备忙时长／观察时长；切换、空闲与有效计算的统计口径按题设。

2.6 调度算法与事件时间线

算法	选择规则	条件及限制
FCFS	就绪到达最早者，通常非抢占	长任务可使短任务等待较久，产生护航效应
SJF	当前就绪者中预计服务时间最短，通常非抢占	需预测时间；持续短任务可使长任务饥饿
SRTF	当前就绪者中剩余时间最短，可抢占	新到达时比较当前剩余量
HRRN	最大 $1 + W/B$，通常非抢占	等待提高响应比，兼顾长短任务
优先级	当前就绪者中优先级最高	数值方向、抢占及同级顺序按题设；老化可缓解饥饿

算法	选择规则	条件及限制
RR	就绪队列轮转，至多运行一个时间片	用完未结束者回队尾；过大片接近 FCFS，过小增加开销
多级队列	任务通常固定所属队列	队间固定优先级或时间比例，队内规则分别规定
多级反馈	根据运行行为迁移队列，常用逐级加长时间片	高层可抢占低层；降级、I/O 返回及剩余片规则按题设

时间线只在已到达且就绪者中选择。记录每次事件的当前执行者、剩余量、各队列顺序；同刻到达与片耗尽的入队次序须明确。CPU 总服务量须与题设一致。最小平均等待等最优性结论需同时到达、已知服务、忽略开销等相应模型前提。

例 2-2

4 个作业到达系统的时刻和运行时间如下表所示。

作业	到达时刻	运行时间
J1	0	3
J2	1	3
J3	1	2
J4	3	1

系统在 $t = 2$ 时开始作业调度。若分别采用先来先服务和短作业优先调度算法，则选中的作业分别是（　）。

A. J2、J3　　B. J1、J4　　C. J2、J4　　D. J1、J3

例 2-3

进程进入就绪队列的时刻、优先级（值越大优先权越高）及 CPU 执行时间如下表所示。

进程	到达	优先级	CPU 时间
P1	0 ms	1	60 ms
P2	20 ms	10	42 ms
P3	30 ms	100	13 ms

采用基于优先权的抢占式 CPU 调度算法，从 0 ms 开始调度，三个进程的平均周转时间为（　）。

A. 60 ms　　B. 61 ms　　C. 70 ms　　D. 71 ms

例 2-4

P0、P1、P2、P3 的到达、优先级（值越小优先权越高）和执行时间如下表所示。采用基于优先权的抢占式进程调度，从 0 ms 开始，到四个进程都运行结束为止，发生进程调度的总次数为（　）。

进程	到达	优先级	CPU 时间
P0	0 ms	15	100 ms
P1	10 ms	20	60 ms
P2	10 ms	10	20 ms
P3	15 ms	6	10 ms

A. 4　　B. 5　　C. 6　　D. 7

例 2-3
执行区间（ms）：P1[0,20)→P2[20,30)→P3[30,43)→P2[43,75)→P1[75,115)。P2 被抢占时还剩 32 ms，P1 还剩 40 ms；P1、P2、P3 的完成时刻分别为 115、75、43，周转时间分别为 $115-0=115$、$75-20=55$、$43-30=13ms$。平均周转 $=(115+55+13)/3=61ms$，选 B。

例 2-4
执行区间（ms）：P0[0,10)→P2[10,15)→P3[15,25)→P2[25,40)→P0[40,130)→P1[130,190)。按本题将 0 ms 首次分派计入、最终转入 idle 不计入的口径，共 6 次调度，选 C；已运行进程之间实际更换执行者 5 次。10 ms 的两个到达事件先共同更新就绪集合，再选 P2，无须逐个到达各计一次分派。

例 2-5

系统采用二级反馈队列调度。Q1 采用时间片轮转，时间片为 10 ms；Q2 采用短进程优先。系统优先调度 Q1，Q1 为空才调度 Q2；新创建进程先进入 Q1，在 Q1 执行一个时间片后仍未结束则转入 Q2。当前两队列为空，系统依次创建 P1、P2 后开始调度，两者需要的 CPU 时间分别为 30 ms、20 ms。平均等待时间为（ ）。

A. 25 ms　　B. 20 ms　　C. 15 ms　　D. 10 ms

例 2-6

系统有 4 个进程，预计运行时间和到达就绪队列的时刻如下，单位均为时间配额。分别采用可抢占式短进程优先和时间片轮转调度（时间片为 2），求各进程的调度次序与平均周转时间。

进程	到达时刻	预计运行时间
P1	0	8
P2	1	4
P3	2	9
P4	3	5

例 2-7

某进程调度程序采用基于优先数（priority）的调度策略，即选择优先数最小的进程运行。进程创建时由用户指定一个 nice 作为静态优先数。为了动态调整优先数，引入运行时间 cpuTime 和等待时间 waitTime，初值均为 0。进程处于执行态时，cpuTime 定时加 1，且 waitTime 置 0；进程处于就绪态时，cpuTime 置 0，waitTime 定时加 1。请回答下列问题。

（1）若调度程序只将 nice 的值作为进程的优先数，即 priority=nice，则可能会出现饥饿现象，为什么？

（2）使用 nice、cpuTime 和 waitTime 设计一种动态优先数计算方法，以避免产生饥饿现象，并说明 waitTime 的作用。

2.7 多处理机与上下文切换

公共就绪队列便于分担负载，访问须互斥；每核私有队列降低争用，需迁移平衡负载。推迁移由系统负载监测程序推动，拉迁移由空闲或较空闲 CPU 的调度程序取任务。软亲和性倾向留在原 CPU，硬亲和性限制允许 CPU 集合；迁移收益须扣除缓存、TLB 局部性和同步成本。SMT 的逻辑处理器共享核内部分执行资源。

上下文包含 PC、PSW、通用寄存器、栈等恢复状态；例如 x++ 在寄存器已加一、尚未写回时被切换，必须保存该寄存器与下一指令位置。模式切换只改变执行权限，可仍执行原进程。

同进程线程切换需换线程现场，通常保留进程页表与文件资源；跨进程还需建立新地址空间环境。TLB 是否清空取决于地址空间标识等机制，普通数据 Cache 无须因进程切换一律全部清空。

2.8 临界区与互斥要求

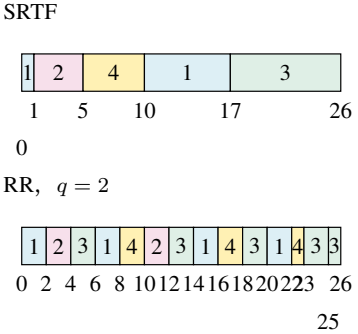
临界资源在同一时期只许一个进程使用；临界区是访问该资源的代码段，进入区申请、退出区释放。多个进程访问同一资源可有多段相关临

例 2-5
以开始调度为 0 ms：Q1 依次执行 P1[0,10)、P2[10,20)，转入 Q2 时两者剩余 20、10 ms；Q2 再执行 P2[20,30)、P1[30,50)。两者均在调度开始前就绪，等待时间为 P1：$50-30=20ms$，P2：$30-20=10ms$，平均 15 ms，选 C。

例 2-6
SRTF 按 P1、P2、P3、P4 排列的完成时刻为 (17,5,26,10)，周转时间为 (17,4,24,7)，平均 13。RR 取 $q=2$，并约定同刻新到达者先入队，则完成时刻为 (22,12,26,23)，周转时间为 (22,11,24,20)，平均 19.25。题干未给同刻入队次序；若 $t=2$ 先让 P1 回队、再入队 P3，平均周转会变为 18.75，须先声明规则再比较答案。图示见合订本第 118、119 页侧栏。

例 2-7
只用 nice 时，较小 nice 的任务可持续占用 CPU，使较大 nice 者一直等待。可取 $priority = nice + cpuTime - waitTime$；例如 nice 为 0、4 的两进程从计数均为 0 开始，一方连续运行、另一方持续等待 t 个单位，其优先数分别为 t 与 4−t，$t>2$ 时等待者优先。等待项提供老化，运行项降低持续占用者的优先权；无饥饿结论还以有限进程、定期重新调度、同级公平及计数不溢出为前提。

例 2-6 · 合订本 118 · 图 1
SRTF 与 RR 执行区间



条内 1～4 代表 P1～P4；时间单位为题设配额。RR 在 $t=25$ 仍保留片界。

界区。同步规定操作先后，互斥限制访问重叠。高级语言 `x=x+1` 的读、加、写可被交错，单核也可能丢失更新。

要求	准确含义
空闲让进	临界区空闲，申请者有进入机会
忙则等待	已有进入者时，其他相关申请者等待
有限等待	请求后被其他进入者越过次数有有限上界；需执行机会和有限临界区等前提
让权等待	无法进入时阻塞，让出 CPU

例 2-8

进程 A 和进程 B 通过共享缓冲区协作完成数据处理，进程 A 负责产生数据并放入缓冲区，进程 B 从缓冲区读数据并输出。进程 A 和进程 B 之间的制约关系是（ ）。

A. 互斥关系　　B. 同步关系　　C. 互斥和同步关系　　D. 无制约关系

例 2-6 · 合订本 118 · 图 2
到达、服务与周转

进程	到达/服务	SRTF 周转	RR 周转
P1	0/8	17	22
P2	1/4	4	11
P3	2/9	24	24
P4	3/5	7	20

同刻新到达者先入队；平均周转 SRTF 为 13、RR 为 19.25。$t=2$ 若先回队 P1，RR 平均改为 18.75。

2.9 软件互斥、原子指令与锁

双进程软件模型假定共享单次读写原子、顺序一致。`i=0,1`，`j=1-i`，`flag` 初值均为假。

方案	进入／退出规则	失效原因
单标志	等 `turn==i`；退出置 `turn=j`	强制交替，对方不申请时也可能阻挡，违背空闲让进
双标志先检查	等 `!flag[j]` 后置 `flag[i]=true`；退出清自身标志	双方可先检查通过再置真，破坏互斥
双标志后检查	先置自身真，再等对方假；退出清自身标志	双方都置真后互等，临界区空闲仍无法进入

Peterson 同时表达意愿与让步：

```
flag[i] = true;
turn = j;
while (flag[j] && turn == j) { }
临界区;
flag[i] = false;
```

Peterson 同时表达意愿与让步
双方均表达申请意愿后，最后写 turn 的一方把优先权让给对方，它的循环条件仍为真，因而双方不能同时进入。等待者保持 flag 为真时，对方退出后若再次申请，会把 turn 写回等待者这一侧；一次申请至多被对方越过 1 次。该上界使用本页的顺序一致、持续调度和有限临界区假设。

在上述内存模型与持续调度、有限临界区条件下，Peterson 满足互斥和有限等待；循环检查仍占 CPU。实际机器需原子变量和适当内存序。

```
// TestAndSet 原子返回旧值并置真；lock 初值 false
while (TestAndSet(lock)) { }
临界区；
lock = false;

// Swap 原子交换；key 为进程私有
key = true;
while (key) Swap(lock, key);
临界区；
lock = false;
```

TS ／ Swap 的检查与占用整体原子；普通赋值展开式须额外原子化。这些自旋实现保证互斥，公平和有限等待还需另设机制。持锁很短且持有者可继续执行时可减少睡眠切换开销；较长等待适合阻塞锁。单核自旋须允许持锁者获得执行机会。本 CPU 关中断仅保护合适的单处理器内核路径，普通用户程序无权执行，亦无法阻挡其他 CPU。

2.10 信号量的数值语义与前驱同步

本章 `P=wait`、`V=signal`，均采用记录型信号量；队列公平性须额外声明。P、V 的计数更新、判断与入队／唤醒须原子化。

```
P(S): S.value--;
      if (S.value < 0) 入队并阻塞;
      // 唤醒后本次 P 返回，不再减 1
V(S): S.value++;
      if (S.value <= 0) 移出一个等待者并置为就绪;
```

`value>0` 为未分配许可数；`value=0` 为无剩余许可且无该量等待者；`value<0` 的绝对值为等待者数。P 减后为 0 可继续；V 加后为 0 仍须唤醒。V 不因缺资源而阻塞，唤醒后仍等 CPU。整型信号量在 `S<=0` 时忙等、再原子减一，通常无负值，需与记录型数值规则分开。

互斥量初值 1；容量 n、初存 k 件时 `empty=n-k, full=k`；未完成通知初值 0。信号量初值来自初态，P 与对应 V 可分属不同进程，通常无互斥锁 API 的持有者限制。

例 2-9

用 P、V 操作实现进程同步，信号量的初值为（　）。

A. -1　　B. 0　　C. 1　　D. 由用户确定

前驱边 $A \to B$ 可用初值 0 的 `done`：前进程 `A; V(done)`，后进程 `P(done); B`。同线程顺序已保证的边无需重复通知；多路汇合须等待每路完成，分叉须分别发布许可。通知放在前操作完成后、后操作开始前，避免增加题设未要求的先后约束。

例 2-10

某进程的两个线程 T1 和 T2 并发执行 A、B、C、D、E 和 F 共 6 个操作，其中 T1 执行 A、E 和 F，T2 执行 B、C 和 D。题 46 图表示上述 6 个操作的执行顺序所必须满足的约束：C 在 A 和 B 完成后执行，D 和 E 在 C 完成后执行，F 在 E 完成后执行。请使用信号量的 wait()、signal() 操作描述 T1 和 T2 之间的同步关系，并说明所用信号量的作用及其初值。

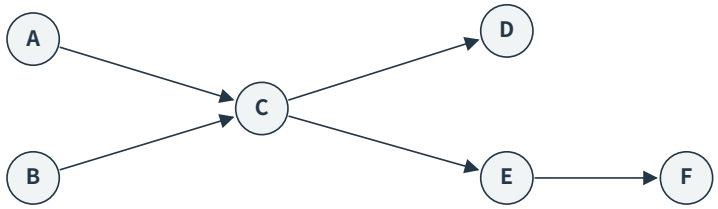


Figure 2.2: 原题前驱图；图下文字标明题干给出的线程归属

2.10 · 原题前驱图；图下文字标明题干给出

只需为跨线程的 A→C、C→E 各设一个初值 0 的信号量 aDone、cDone：T1：A; V(aDone); P(cDone); E; F; T2：B; P(aDone); C; V(cDone); D; aDone 只在 A 完成后发布，cDone 只在 C 完成后发布；B→C、C→D、E→F 由各线程自身顺序保证，D 与 E 可并发。两个方向不能直接共用同一个普通信号量，否则 T1 可能在 A 之后取回自己刚发布的许可，使 E 抢在 C 之前执行。

2.11 生产者与消费者

容量 $n > 0$，初始为空，生产和消费可反复进行，每次放取一件；共享缓冲及指针须互斥。局部生产、使用产品放在锁外。

```
semaphore empty=n, full=0, mutex=1;
producer: while (true) {
    x=produce();
    P(empty); P(mutex);
    put(x);
    V(mutex); V(full);
}
consumer: while (true) {
    P(full); P(mutex);
    x=get();
    V(mutex); V(empty);
    consume(x);
}
```

本模板先取得容量／产品许可，再取得缓冲锁。空缓冲时若先持 `mutex` 再等 `full`，生产者无法放入，形成死锁。放入完成后两次 V 可交换，条件是消费者仍须持同一锁访问；这种可交换性依赖具体代码。

许可可能已预留或尚未发布，记录型值还可为负；逐指令执行时 `empty+full=n` 与 “full 等于实存产品数” 均不恒成立。静止且无预留、未发布或等待者时，才能直接对应物理数量。

许可可能已预留或尚未发布

令 $E = \max(empty, 0)$、$F = \max(full, 0)$，Rp 计已获得 empty 许可但尚未执行 V(full) 的生产者，Rc 计已获得 full 许可但尚未执行 V(empty) 的消费者，则 $E + F + Rp + Rc = n$。获许可包括已被 V 定向唤醒、尚未得到 CPU 者；仍在等待队列者不计入 Rp、Rc。按 P、V 整体原子操作的边界计数，并假定每次交接均正常完成；例如初始空缓冲 $n = 2$，一名生产者刚预留空位时，$(E, F, Rp, Rc) = (1, 0, 1, 0)$。

例 2-11

计算机系统中的进程之间往往需要相互协作以完成一个任务。在某网络系统中，缓冲区 B 用于存放一个数据分组，对 B 的操作有 C1、C2 和 C3。C1 将一个数据分组写入 B 中，C2 从 B 中读出一个数据分组，C3 对 B 中的数据分组进行修改。要求 B 为空时才能执行 C1，B 非空时才能执行 C2 和 C3。

（1）假设进程 P1 和 P2 均需要执行 C1，实现 C1 的代码是否为临界区？为什么？（2 分）

（2）假设 B 初始为空，进程 P1 执行 C1 一次，进程 P2 执行 C2 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3 分）

（3）假设 B 初始不为空，进程 P1 和 P2 各执行 C3 一次。请定义尽可能少的信号量，并用 wait()、signal() 操作描述进程 P1 和 P2 之间的同步或互斥关系，说明所用信号量的作用及其初值。（3 分）

分类产品分别设置可消费许可，共用空位和缓冲锁；连续取 k 件需消费

例 2-11

（1）C1 是访问同一缓冲 B 的临界区：两个写入者须互斥检查并占用空缓冲，防止覆盖彼此的数据。

（2）仅执行各一次，设 $ready = 0$：P1 为 C1; V(ready)，P2 为 P(ready); C2；这一先后关系已使两次访问不重叠，只需一个信号量。

（3）B 初始非空且 C3 不取走分组，设 $mutex = 1$；两进程均执行 P(mutex); C3; V(mutex)，同样只需一个信号量。第（2）问的简化依赖 “一次”，若反复生产消费，还须保证下一次写入等待上一分组被取走。

者批锁覆盖整批，单件缓冲锁只覆盖放取，等待产品时保留生产者补货通路。

例 2-12

三个进程 P1、P2、P3 互斥使用一个包含 N（N>0）个单元的缓冲区。P1 每次用 produce() 生成一个正整数并用 put() 送入缓冲区某一空单元中；P2 每次用 getodd() 从该缓冲区中取出一个奇数并用 countodd() 统计奇数个数；P3 每次用 geteven() 从该缓冲区中取出一个偶数并用 counteven() 统计偶数个数。请用信号量机制实现这三个进程的同步与互斥活动，并说明所定义信号量的含义（要求用伪代码描述）。

例 2-13

系统中有多个生产者进程和多个消费者进程，共享一个能存放 1000 件产品的环形缓冲区（初始为空）。当缓冲区未满时，生产者进程可以放入其生产的一件产品，否则等待；当缓冲区未空时，消费者进程可以从缓冲区取走一件产品，否则等待。要求一个消费者进程从缓冲区连续取出 10 件产品后，其他消费者进程才可以取产品。请使用信号量 P、V（或 wait()、signal()）操作实现进程间的互斥与同步，要求写出完整的过程，并说明所用信号量的含义和初值。

2.12 单盘水果模型

盘子容量 1，初始空；父亲放苹果、母亲放橘子，女儿取苹果、儿子取橘子：

```
semaphore plate=1, apple=0, orange=0;
father:   while (true) { 准备苹果; P(plate); 放苹果; V(apple); }
mother:   while (true) { 准备橘子; P(plate); 放橘子; V(orange);
    }
daughter: while (true) { P(apple); 取苹果; V(plate); 吃苹果; }
son:      while (true) { P(orange); 取橘子; V(plate); 吃橘子; }
```

该容量及交接规则使访问权始终由一个活动持有，同步兼顾盘子互斥。容量增加后，多个生产者可同时取得空位，共享结构还需 `mutex`；在实际放取动作周围加锁。

2.13 读者与写者

多个读者可并发读；写者与读者、写者之间互斥。`count` 记录已登记且未退出的读者，首读者为全组申请 `rw`，末读者归还，`cm` 保护计数及首末判断。

例 2-12
假定初始为空，设 $empty = N$、$odd = 0$、$even = 0$、$mutex = 1$，x 为各进程局部变量：
P1：while(true){ x=produce(); P(empty); P(mutex); put(x); V(mutex); if(x%2) V(odd); else V(even); }
P2：while(true){ P(odd); P(mutex); x=getodd(); V(mutex); V(empty); countodd(); }
P3：while(true){ P(even); P(mutex); x=geteven(); V(mutex); V(empty); counteven(); }
odd、even 分别发放对应产品的取用许可，始终有 $0 \le$ 实存奇数数 +实存偶数数 $\le N$；等待分类许可时未持 mutex，生产者可继续放货。题干未写初始存量；若初存 o 个奇数、e 个偶数，则改为 $empty = N - o - e$、$odd = o$、$even = e$，且 $0 \le o + e \le N$。

例 2-13
设 $empty = 1000$、$full = 0$、$mutex = 1$、$batch = 1$，环形下标 $in = out = 0$；x 和数组 b[10] 均为进程局部数据。
producer：while(true){ x=produce(); P(empty); P(mutex); B[in]=x; in=(in+1)%1000; V(mutex); V(full); }
consumer：while(true){ P(batch); for(j=0;j<10;j++){ P(full); P(mutex); b[j]=B[out]; out=(out+1)%1000; V(mutex); V(empty); } V(batch); consume(b); }
batch 保证同一时刻只有一个消费者领取整批，mutex 仅保护单件放取及下标，库存始终在 0 至 1000 之间。消费者等 full 时仍可由生产者补货；生产者不申请 batch。若后续产品不足以凑齐 10 件，该批可继续等待，完成一批的活性以最终有足够产品为前提。

```
semaphore rw=1, cm=1;
int count=0;
reader() {
    P(cm);
    if (count==0) P(rw);
    count++;
    V(cm);
    读文件;
    P(cm);
    count--;
    if (count==0) V(rw);
    V(cm);
}
writer() { P(rw); 写文件; V(rw); }
```

检查首末与计数修改必须在同一临界区。首读者持 cm 等 rw 时 count=0，无旧读者退出依赖 cm，写者释放也无需 cm。本方案读者优先，连续读者可使写者饥饿。

加入入口 gate=1：读者在上述登记入口前 P(gate)、登记后 V(gate)；写者 P(gate); P(rw); 写; V(rw); V(gate)，读者退出保持原样。写者取得 gate 后阻止新读者登记，旧读者可退出；FIFO 等公平排队、持续调度及有限读写共同支持无饥饿结论，普通信号量本身无此保证。

2.14 哲学家进餐

五人 P_i 需同时持有 C_i 与 $C_{(i+1) \bmod 5}$，各筷子初值 1。全员先取同侧会形成五人各持一根的等待环。限制最多四人进入申请到释放区间：

限制最多四人进入申请到释放区间
若出现全体停滞，持筷者都只能拿到第一根并等待下一根；Pi 所等的 C(i+1) 须由下一位作为第一根持有。沿环反复推出五人全在申请区，与同时至多 4 人矛盾，所以本方案排除了环形死锁。这个论证只保证有人能够推进，个别人的有限等待仍需原文所述公平条件。

```
semaphore room=4, C[5]={1,1,1,1,1};
philosopher(i) {
    while (true) {
        思考;
        P(room); P(C[i]); P(C[(i+1)%5]);
        吃饭;
        V(C[(i+1)%5]); V(C[i]); V(room);
    }
}
```

四人是申请人数上限，五根筷子最多支持两人同时进餐。其他无环方案：全局按资源编号从小到大申请；用额外互斥量串行化拿两根的过程，拿齐后释放该量，吃完直接归还筷子。所有方案的无饥饿仍需公平条件；各人独立先左后右无法形成全局递增次序。

2.15 管程与条件变量

管程封装共享数据、操作及初始化，同一时刻至多一个进程在其中执行，由语言或运行系统保证入口互斥。条件变量管理等待队列，布尔条件由共享状态表示。执行 x.wait() 即入队阻塞并释放管程；x.signal() 通知一个等待者，无人等待时不积累许可。信号量 V 则会留下计数。

例 2-14

若 x 是管程内的条件变量，则当进程执行 x.wait() 时所做的工作是（　）。

A. 实现对变量 x 的互斥访问

B. 唤醒一个在 x 上阻塞的进程

C. 根据 x 的值判断该进程是否进入阻塞态

D. 阻塞该进程，并将之插入 x 的阻塞队列中

Hoare 语义可立即把管程交给被通知者；Mesa 语义允许通知者继续，等待者以后重新竞争。Mesa ／ POSIX 式等待应使用 `while`：

```
put(x) {
    lock(m);
    while (count==n) wait(notFull,m);
    插入x; count++;
    signal(notEmpty); unlock(m);
}
get() {
    lock(m);
    while (count==0) wait(notEmpty,m);
    x=取出; count--;
    signal(notFull); unlock(m);
    return x;
}
```

`wait` 原子释放锁并入队，返回前重新取得锁；重检处理竞争与可能的虚假唤醒。拆开 “解锁、入队” 会丢失中间通知。Java 同步实例方法锁定相应对象，静态同步方法锁定类对象；多个实例的锁通常分别存在。

2.16 死锁条件、资源图与安全性

以下讨论可重复使用资源，完成时归还；最大需求可信、可满足后可有限完成。死锁是一组进程各等组内才能产生的事件而无法自行推进；饥饿是个别进程长期得不到服务、其他进程仍可推进；死循环是执行路径持续重复。非重入锁可发生单进程自我死锁。

死锁必要条件	内容
互斥	实例同一时刻只给一个使用者
占有并等待	保持已有资源，再等新增资源
不可剥夺	使用中的资源不能被任意收回
循环等待	存在首尾相接的资源等待链

资源分配图中，进程指向资源为请求边，资源实例指向进程为分配边。无环可排除此模型的死锁；有环且相关资源每类单实例时足以判死锁，多实例时须化简。

安全序列保证每个进程的最大剩余需求可由当时可用量及前者归还量满足。安全表示存在至少一条完整序列；序列可不唯一，也无须强制按其执行。死锁必不安全，不安全可能尚未死锁。

死锁必不安全，不安全可能尚未死锁

设同类资源共 3 个，两个进程各声明最多需 3 个、各已持 1 个，$Available = 1$，而两者 Need 均为 2，没有安全序列。若当前两者均无未满足的请求，检测仍可依次化简，当前尚未死锁；若随后两者都申请余下 2 个，并保持已有资源等待，则陷入死锁。安全性考察未来最大需求，当前死锁考察已发生的等待关系。

2.17 死锁预防、避免与解除

策略	实施方法	条件或代价
预防互斥条件	可共享资源改按共享方式组织；SPOOLing 提供逻辑打印服务	物理打印仍串行，固有互斥语义须保留
预防占有并等待	一次申请全部，或申请失败先释放已有资源	需知需求、可能降低利用率
允许剥夺	保存恢复资源状态并回滚	适合可恢复资源，纸上输出难撤回
预防循环等待	统一编号，严格递增申请	所有参与者须遵循同一全局次序
避免	每次拟分配后检查安全性	需最大需求等未来信息，可能推迟现有量足够的请求
检测及解除	允许进入死锁，检测后终止、剥夺或回滚	考虑已做工作、恢复成本、优先级；反复同一牺牲者可饥饿

例 2-15

若系统 S1 采用死锁避免方法，S2 采用死锁检测方法。下列叙述中，正确的是（ ）。

I. S1 会限制用户申请资源的顺序，而 S2 不会

II. S1 需要进程运行所需的资源总量信息，而 S2 不需要

III. S1 不会给可能导致死锁的进程分配资源，而 S2 会

A. 仅 I、II　　B. 仅 II、III　　C. 仅 I、III　　D. I、II、III

仅降低死锁者优先级无法使其释放资源。预防改变请求规则，避免依据当前状态和最大需求，检测依据当前等待请求。

2.18 银行家算法

向量按同类资源逐项比较。`Need=Max-Allocation`，`Available` 为当前未分配量；始终满足总资源守恒。

```
Work=Available; Finish[0..n-1]=false;
while 存在 i 满足 !Finish[i] && Need[i]<=Work:
    Work += Allocation[i];
    Finish[i]=true;
全部标记：安全；仍有未标记且无可选者：不安全。
```

模拟中借出的 Need 随完成归还相抵，净增为原 Allocation。检查新请求 Q_i：① $Q_i \le Need_i$；② $Q_i \le Available$；③ 试减 Available、试增 Allocation、试减 Need，完整执行安全检查，安全则提交，否则回滚并等待。

例 2-16

某时刻进程的资源使用情况如下表所示，可用资源向量为 $(0,2,1)$。

进程	已分配 (R_1,R_2,R_3)	尚需分配 (R_1,R_2,R_3)
P1	(2,0,0)	(0,0,1)
P2	(1,2,0)	(1,3,2)
P3	(0,1,1)	(1,3,1)

进程	已分配 (R_1, R_2, R_3)	尚需分配 (R_1, R_2, R_3)
P4	(0,0,1)	(2,0,0)

此时的安全序列是（　）。

A. P1、P2、P3、P4　　B. P1、P3、P2、P4

C. P1、P4、P3、P2　　D. 不存在

例 2-16

初始 $Work = (0, 2, 1)$，只有 P1 满足 $Need \le Work$；完成后 $Work = (2, 2, 1)$，此时只能继续选 P4，得到 $Work = (2, 2, 2)$。剩下 P2、P3 均还需 3 个 R2，而 Work 中只有 2 个 R2，无法继续化简，因此不存在完整安全序列，选 D。两次可行前缀相同且均无分支，不能通过改换排序消除阻塞。

例 2-17

系统有三类资源 A、B、C 和五个进程，某时刻分配如下。若系统安全，则可用资源向量 (x, y, z) 可能是哪些?

进程	Allocation	Max
P0	(0,0,3)	(0,0,4)
P1	(1,0,0)	(1,7,5)
P2	(1,3,5)	(2,3,5)
P3	(0,0,2)	(0,6,4)
P4	(0,0,1)	(0,6,5)

I. (1,4,0)　II. (0,6,2)　III. (1,1,1)　IV. (0,4,7)

A. I、II、IV　　B. I、II　　C. 仅 I　　D. I、III

单资源模型总量 100，B、A、T 的 Max 为 70、40、50，Allocation 为 20、10、30，可用 40。给 B 再分 30 后可用 10、Need 为 20、30、20，无人可先完成，应推迟；独立从原态给 A 再分 20 后可用 20、Need 为 50、10、20，可取 T→B→A，Work 为 20→50→70→100。

若 n 个进程每个最多需 $k \ge 1$ 个同类资源，最坏全员各持 $k-1$ 个仍未满足；总量

$$m \ge n(k-1)+1$$

足以保证任意分配顺序下至少一人可拿齐并完成。它属于任意分配的总量保证；银行家可通过推迟不安全请求在更少资源下安全运行。

例 2-17

Need 按 P0 至 P4 排列为 (0,0,1)、(0,7,5)、(1,0,0)、(0,6,2)、(0,6,4)。

I 可取 P2→P0→P1→P3→P4，Work 依次为 (1,4,0)→(2,7,5)→(2,7,8)→(3,7,8)→(3,7,10)→(3,7,11)。

II 完成 P0、P3、P4 后只能到 (0,6,8)：P1 缺 B，P2 缺 A；III 完成 P0、P2 后到 (2,4,9)，其余进程均至少还需 6 个 B；IV 完成 P0 后为 (0,4,10)，其余进程仍不满足。故只有 I 安全，选 C；能启动某个进程只证明存在可行前缀。

2.19 多实例资源的死锁检测

检测用当前未满足 `Request`，避免用未来最大 `Need`。设 `Work=Available`，通常先将 Allocation 全零者标记完成；反复选择未标记且 `Request[i]<=Work` 者，令 `Work+=Allocation[i]` 并标记。最终持有资源且无法化简者构成死锁核心；无持有量的等待者也可能受其影响，集合范围按题用定义。

设 R_1 共 3、R_2 共 2：P1 持 $(2,0)$，P2 持 $(1,1)$ 并请求 $(1,0)$。若 P1 请求 $(0,1)$，初始 Work=$(0,1)$，可先 P1 后 P2 完成；若另一 R_2 由无请求的 P3 持有，P1 请求 $(0,2)$，P3 归还后 Work 仅 $(0,1)$，P1、P2 仍无法化简。